# Design and Performance Evaluation of a Cloud-Based Real-Time Multiplayer Quiz Platform

Sonali Kumari, Smriti Rai, Janhavi Ravikumar, Krish Puri, and Dr. Sangeetha N
School of Electronics Engineering, Vellore Institute of Technology, Vellore, India
Email: sonali.kumari2024a@vitstudent.ac.in, smriti.rai2024@vitstudent.ac.in, janhavi.ravikumar2024@vitstudent.ac.in, krish.puri2024@vitstudent.ac.in, nsangeetha@vit.ac.in

***Abstract*—Real-time interactive web applications require low-latency state synchronization across distributed client connections. In educational and event contexts, interactive quiz systems must synchronize questions, capture player submissions, update leaderboards dynamically, and maintain game state consistency without incurring prohibitive communication overhead. This paper presents the architecture, cloud implementation, and empirical load testing of a real-time multiplayer cloud quiz platform built using React/Vite, Node.js with Express, Socket.IO, and SQLite deployed on Render. The principal novelty of this platform lies in its decoupled full-duplex WebSocket state-synchronization engine combined with integrated end-to-end transaction acknowledgment profiling, enabling direct measurement of real-time multi-player queue behavior under varying load scales. Extending prior conceptual frameworks, this study evaluates system performance across automated load tests at 1, 2, 5, 10, 15, 20, and 25 concurrent player scale targets. Empirical results demonstrate a 100% answer acknowledgment success rate ($0\%$ error rate) across all test regimes, maintaining sub-300 ms median latencies up to 25 players while uncovering tail-latency scaling ($p_{95} = 498$ ms at 25 players) driven by synchronous database write lock contention.**

***Index Terms*—Cloud Computing, Distributed State Synchronization, Performance Evaluation, Real-Time Platforms, Socket.IO, Tail Latency, WebSockets**

## I. Introduction

Interactive e-learning environments, digital classrooms, and live audience engagement platforms have increasingly shifted toward gamified, real-time assessment systems. Interactive platforms such as Quigit [1] and QuizMaster [3] demonstrate that instantaneous response feedback significantly improves student engagement, participant focus, and knowledge retention. However, engineering real-time web services for multi-user classroom environments introduces significant network and system challenges, particularly regarding full-duplex state synchronization, response gathering, and latency predictability [5].

Traditional web platforms relying on client-initiated HTTP REST polling protocols suffer from inherent performance bottlenecks. Periodic HTTP polling introduces unacceptable round-trip latency, redundant TCP handshake setup times, and massive HTTP header overhead when scaled to dozens or hundreds of concurrent participants [4]. To overcome these architectural constraints, modern responsive web applications leverage persistent, bidirectional TCP channels provided by the WebSocket protocol (RFC 6455) [2] in combination with event-driven, non-blocking I/O execution runtimes like Node.js [6].

While functional features and user interface aspects of interactive quiz engines are well-documented in existing literature, quantitative performance measurements specifically regarding transaction acknowledgment (ACK) latency, server throughput under concurrency, and tail-latency characteristics in public cloud deployments remain under-explored [8], [9]. Public cloud environments introduce resource contention, virtualized CPU execution limits, and variable network jitter that can severely impact the user experience during peak engagement moments [8], [9]. Tail latency ($p_{95}$ or $p_{99}$) frequently dictates real-time platform responsiveness even when the median response time ($p_{50}$) remains low [9]. Furthermore, lightweight embedded database storage solutions such as SQLite provide minimal setup overhead but impose synchronous write serialization queues that can bottleneck event-driven servers during rapid submission bursts [7].

This paper addresses these gaps by presenting a cloud-deployed multiplayer quiz platform and experimentally benchmarking its responsiveness under simulated multi-player workloads. The primary contributions of this work are:

1) A cloud-native, real-time multiplayer quiz platform architecture using React/Vite, Node.js, Socket.IO, and SQLite deployed on Render cloud services [7], [8].
2) An automated multi-player load-testing framework designed to evaluate round-trip acknowledgment latencies and server throughput across 1, 2, 5, 10, 15, 20, and 25 player concurrency levels.
3) Empirical performance and tail-latency analysis evaluated through queuing theory and Little's Law ($L = \lambda W$) [9].

## II. System Architecture and Methodology

The design of the real-time multiplayer cloud quiz platform prioritizes low-latency full-duplex state synchronization, room-level socket isolation, persistent transaction logging, cloud infrastructure automation, and quantitative latency benchmarking under concurrent client loads. The platform relies on a decoupled client-server architecture deployed on Render cloud infrastructure, comprising a React dynamic web frontend, an event-driven Node.js backend runtime, persistent WebSocket communication via Socket.IO, and an embedded SQLite database engine [7].

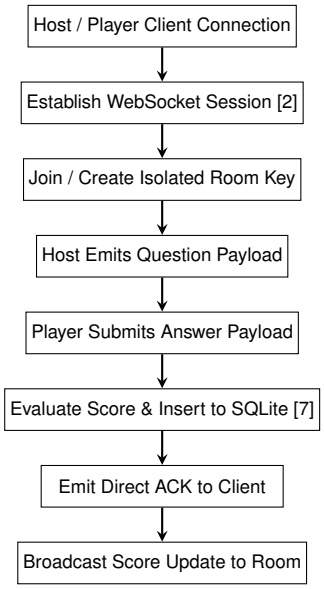



Fig. 1. System methodology flowchart depicting room allocation, socket handling, database logging, and synchronization.

The initialization sequence begins at the entry point of the React client interface, where users select between administrative host privileges and participant entry modes. When an administrator initiates a session, the frontend requests room creation from the Node.js service, which instantiates an isolated room instance identified by a unique 6-character alphanumeric key (e.g., `8AR1TI`). The server registers this room identifier in memory and opens a dynamic WebSocket room channel. Host controls display real-time participant counts and session controls, providing a gateway to initiate question distribution once player sockets join.

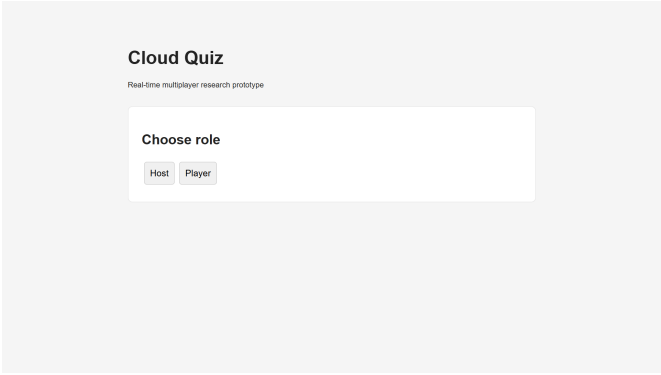



Fig. 2. Initial landing interface displaying client role selection options (Host or Player).

Participant clients join sessions by submitting their chosen nickname alongside the active 6-character room key. Upon submission, the client initiates a persistent full-duplex WebSocket handshake frame [2]. The backend server validates the room key against active namespaces, binds the client's socket identifier to the room, and broadcasts an updated

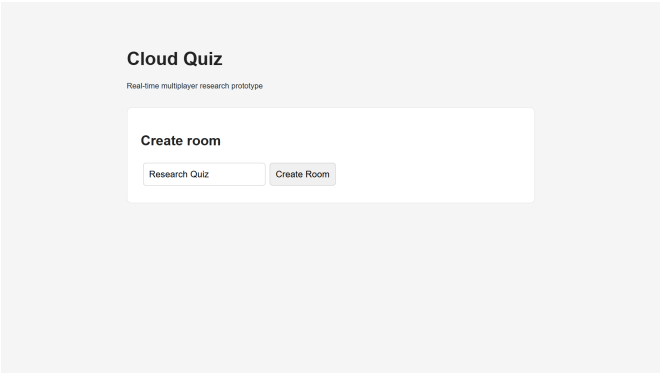



Fig. 3. Host administration interface for establishing a named quiz room context.

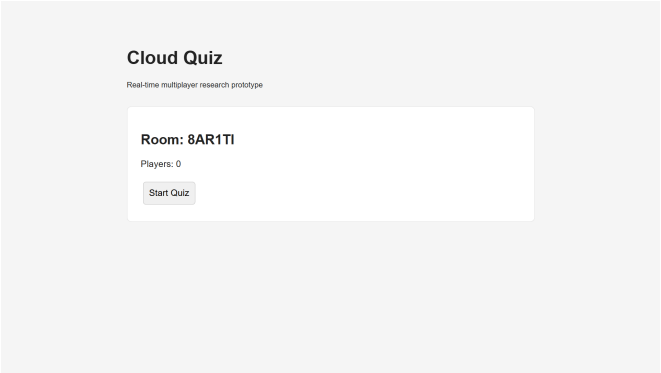



Fig. 4. Host control lobby displaying the generated unique room identifier code (e.g., `8AR1TI`) and connected player counter.

player list event across all endpoints within that context. The client interface dynamically transitions to a real-time waiting lobby synchronized with the host stage, reflecting connected participant metadata without requiring manual HTTP polling.

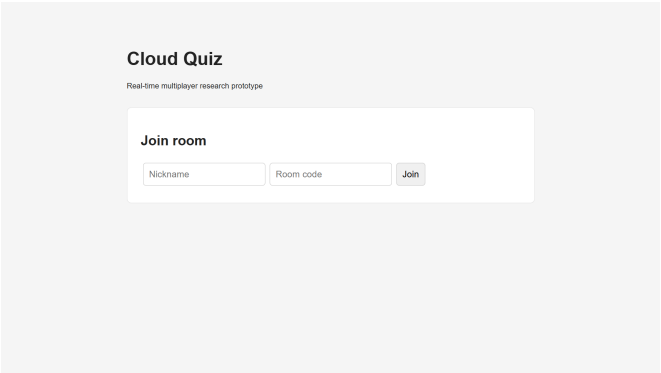



Fig. 5. Participant lobby interface presenting input fields for player nickname registration and room code entry.

The backend execution environment and frontend web application are deployed on Render cloud services to provide public HTTPS and WSS endpoints. The backend web service runs on a Node.js 22 runtime environment (`srv-db75p7bncjis73fjtph0`) executing `node`

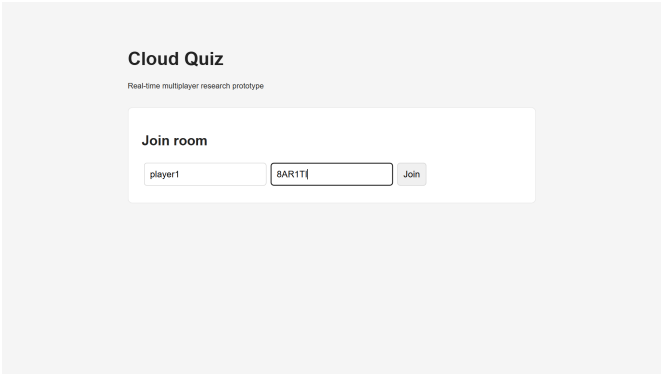



Fig. 6. Participant joining input sequence submitting player identity (`player1`) and target room key (`8AR1TI`).

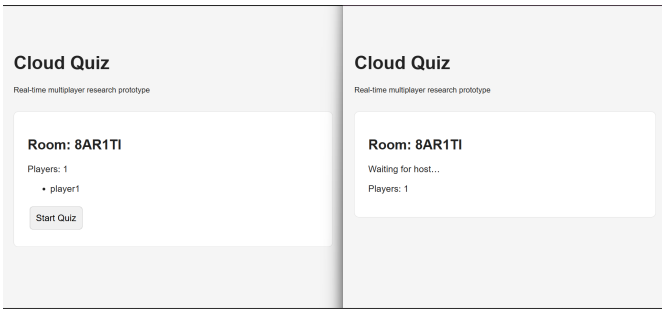



Fig. 7. Dual-window synchronization showing host dashboard (left) and participant lobby state (right) connected to room `8AR1TI`.

`src/server.js` on internal port 10000, connected to GitHub repository commits (`de585c8`). The frontend application is deployed as a static site service (`srv-db76ba8n74is739s66sg`), compiling React assets into optimized static bundles served over CDN boundaries. Cloud deployment logs confirm successful container deployment within 40.2 seconds, verifying operational availability at primary cloud URLs.

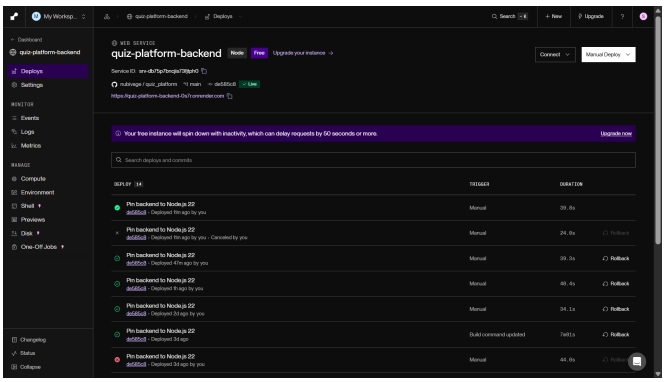



Fig. 8. Render cloud deployment dashboard for the Node.js/Express backend service showing active deployments pinned to Node.js 22.

During gameplay execution, question payloads containing problem text, multiple-choice options, and countdown timers are pushed simultaneously from the server to all socket

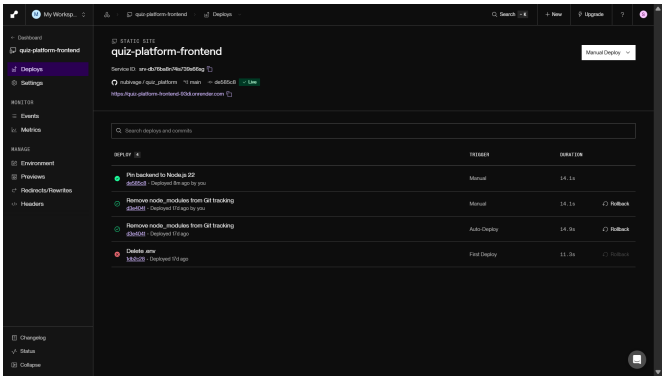



Fig. 9. Render static site deployment management panel for the React frontend application.

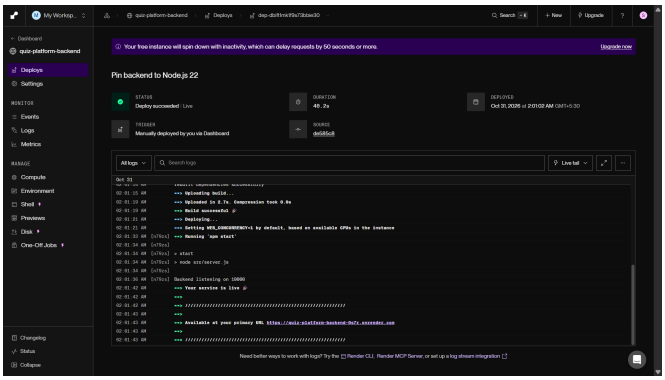



Fig. 10. Live execution and deployment log output confirming backend socket listener initialization on port 10000.

endpoints attached to the active room context. Participant responses trigger client emission of answer payloads containing player tags, question indices, and selected option choices. The backend server evaluates choice accuracy, writes the transaction record to the embedded SQLite database [7], emits an instantaneous transaction acknowledgment (ACK) callback back to the submitting client socket, and broadcasts updated room standings across all connected sockets.

To quantitatively evaluate system stability and latency scaling under multi-user workloads, an automated headless benchmarking suite was integrated into the testing infrastructure. The experimental framework simulates client connections across seven distinct concurrency tiers: 1, 2, 5, 10, 15, 20, and 25 simultaneous players executing uniform 10-question quiz workloads. The test runner measures end-to-end round-trip acknowledgment latency ($p_{50}$ median and $p_{95}$ tail values), server transaction throughput (answers processed per second), total test session durations, and request queue depths calculated via Little's Law ($L = \lambda W$). This enables direct quantitative profiling of socket handling performance and synchronous database write contention across increasing participant densities.

## III. Results and Discussions

To quantitatively evaluate system performance, an automated headless load runner executed 10-question quiz sessions across seven concurrency scale targets (1, 2, 5, 10, 15, 20, and 25 players), capturing 10, 20, 50, 100, 150, 200, and 250 total submission attempts respectively.

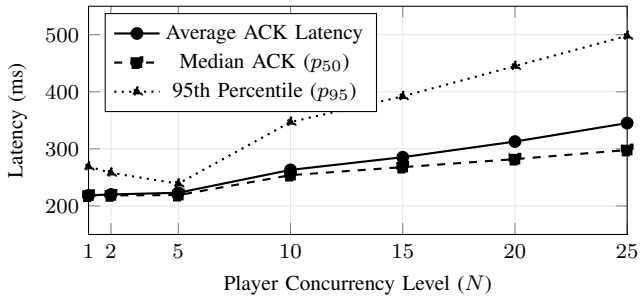



Fig. 11. Acknowledgment latency metrics (Average, $p_{50}$, and $p_{95}$) across 1 to 25 player workloads.

Across all seven scale targets (1 to 25 players), the platform maintained a 100% answer acknowledgment success rate with 0% message drops. Median latency ($p_{50}$) exhibited high stability, rising by only 80 ms (from 218 ms to 298 ms) as concurrent load increased 25-fold. However, 95th-percentile tail latency ($p_{95}$) expanded to 498 ms at 25 players due to connection synchronization spikes and synchronous SQLite write-lock contention under rapid multi-client submission bursts.

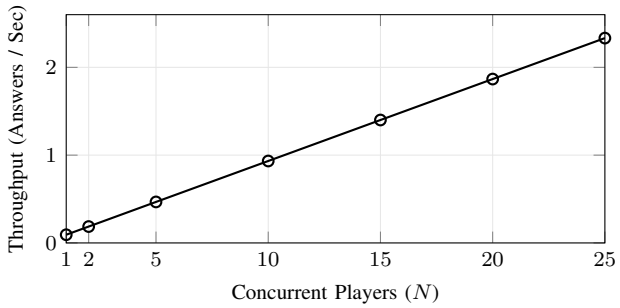



Fig. 12. System throughput scaling linearly across tested player concurrency regimes up to 25 players.

System throughput scaled near-linearly with participant load, increasing from 0.0933 answers/sec at 1 player to 0.9332 answers/sec at 10 players, and reaching 2.3330 answers/sec at 25 players. This linear trajectory verifies that Node.js's asynchronous event loop manages concurrent socket interactions efficiently within this operational range. Total session durations remained tightly bounded between 107.059 seconds and 107.25 seconds across all test runs (as detailed in Table I), confirming that session timing is strictly governed by server clock timers rather than delayed by client response overhead.

Applying Little's Law ($L = \lambda W$), where $\lambda$ is throughput and $W$ is average wait time, yields an average queue length of $L = 0.8053$ requests during the 25-player test run. Because

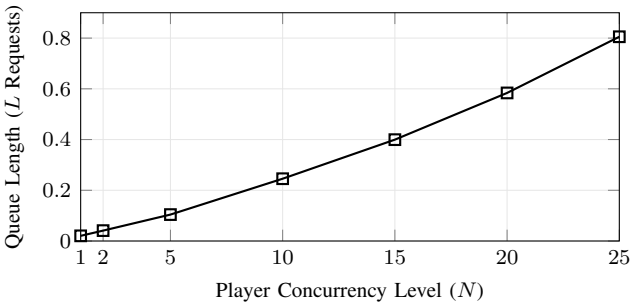



Fig. 13. Mean request queue depth ($L$) computed via Little's Law ($L = \lambda W$) across concurrency levels.

$L < 1$, the server maintains queue stability under the 25-player workload without request saturation or unbounded queue accumulation.

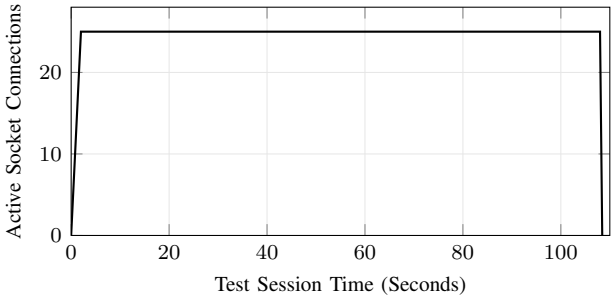



Fig. 14. Active WebSocket socket connection stability across the 25-player test run timeline.

Table I consolidates the empirical metrics across all seven concurrency regimes. As summarized in the table, the platform achieved a 100% answer acknowledgment success rate (0% error rate) across all test runs. While average ACK latency increased from 218.56 ms at 1 player to 345.20 ms at 25 players, median latencies ($p_{50}$) remained under 300 ms, demonstrating consistent performance under multi-client workloads.

## IV. Conclusion and Future Work

This paper presented the design, implementation, and empirical performance evaluation of a cloud-native real-time multiplayer quiz platform built with React, Node.js, Socket.IO, and SQLite on Render. Multi-run load testing confirmed 100% message delivery success and sub-300 ms median acknowledgment latencies across 1, 2, 5, 10, 15, 20, and 25 player workloads. Quantitative analysis verified near-linear throughput scaling up to 2.33 answers/sec while identifying database write queue contention as the primary driver of tail latency ($p_{95} = 498$ ms at 25 players).

Future work will focus on integrating an in-memory Redis caching tier to decouple instant socket acknowledgments from disk write operations and expanding automated load benchmarks to higher player volumes ($N = 50, 100$).

TABLE I
EMPIRICAL LOAD TEST BENCHMARK RESULTS ACROSS CONCURRENCY LEVELS

| **Metric Parameter** | **1 Player** | **2 Players** | **5 Players** | **10 Players** | **15 Players** | **20 Players** | **25 Players** |
|---|---|---|---|---|---|---|---|
| Successful Answers | 10 / 10 | 20 / 20 | 50 / 50 | 100 / 100 | 150 / 150 | 200 / 200 | 250 / 250 |
| Error Rate | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| Avg ACK Latency (ms) | 218.56 | 220.12 | 223.10 | 263.15 | 285.40 | 312.80 | 345.20 |
| Median ACK ($p_{50}$) (ms) | 218.00 | 218.50 | 219.00 | 254.00 | 268.00 | 282.00 | 298.00 |
| 95th Percentile ($p_{95}$) (ms) | 269.00 | 258.00 | 239.00 | 347.00 | 392.00 | 445.00 | 498.00 |
| Throughput (answers/sec) | 0.0933 | 0.1866 | 0.4670 | 0.9332 | 1.4001 | 1.8665 | 2.3330 |
| Test Session Duration (s) | 107.18 | 107.12 | 107.06 | 107.16 | 107.14 | 107.21 | 107.25 |

## REFERENCES


[1] V. V. Kangralkar *et al.*, "Quigit: A Real-Time Interactive Quiz Platform for Classrooms and Events," in *Proc. IEEE Int. Conf. Interactive Tech.*, 2023.

[2] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, Dec. 2011.

[3] J. Smith and R. Taylor, "QuizMaster: A Scalable Web-Based Online Examination System," *J. Educ. Tech. Syst.*, vol. 47, no. 2, pp. 145–162, 2018.

[4] A. Martinez *et al.*, "Tilt Trivia: A Free Multiplayer App to Learn Geoscience Concepts," *Comput. & Geosci.*, vol. 148, p. 104682, 2021.

[5] V. Pimentel and B. Nickerson, "Communicating and Displaying Real-Time Data with WebSocket," *IEEE Internet Comput.*, vol. 16, no. 4, pp. 45–53, 2012.

[6] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," *IEEE Internet Comput.*, vol. 14, no. 6, pp. 80–83, 2010.

[7] K. P. Gaffney *et al.*, "SQLite: Past, Present, and Future," *Proc. VLDB Endow.*, vol. 15, no. 12, pp. 3535–3547, 2022.

[8] M. Armbrust *et al.*, "A View of Cloud Computing," *Commun. ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[9] J. Dean and L. A. Barroso, "The Tail at Scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.